

UndoScope: Effect-Scoped Compensation for Tool-Using Agents

Shivam Gupta

Independent Research

shivam1720406@gmail.com

23 September 2026

Abstract

An agent can select the correct recovery tool and still damage another actor’s work. A mapped inverse may overwrite an intervening assignment, remove an independently granted permission, or delete a resource that has changed since creation. We study *recovery-effect isolation*: compensation may remove the recorded effect of an authorized workflow while preserving independent effects committed afterward. UNDOSCOPE combines established compensation, provenance, version checks, and runtime authorization in a single-store reference contract for register assignments, additive contributions, tagged grants, and resource creation. Its contribution is an executable security specification and benchmark, rather than a new rollback primitive. An indistinguishability argument explains why current-value comparisons cannot provide both useful recovery and preservation of same-value peer writes. Across 1,364 bounded event histories, mapped inverses cause 521 collateral outcomes; UNDOSCOPE causes none and completes all 678 recoveries admitted by the specified oracle. Object and component guards also avoid collateral mutation but complete 4 and 42 eligible recoveries. In 240 controlled model recovery tasks, paired execution of the same selected calls produces 60 collateral outcomes with mapped inverses and none with UNDOSCOPE. Hardened GPT-4.1 selects no wrong receipts in its 60 tasks, yet mapped recovery still causes 15 collateral changes. Persistent SQLite experiments exercise concurrent retries and process death, and a working MCP adapter exposes only receipt-based recovery. The results identify operation-specific recovery semantics as a security boundary distinct from prompt robustness, with explicit limits for dependent workflows and external services.

1 Introduction

A support agent changes a ticket from **open** to **triage**. A downstream task fails, so the agent tries to undo its earlier change. Meanwhile, a support engineer has reviewed the ticket and independently confirmed **triage**. A compensator that checks whether the ticket still says **triage** and then restores **open** erases the engineer’s decision. The receipt is genuine, the target is correct, and the inverse parameter is exactly the old value. The failure is in the semantics of recovery.

A similar ambiguity appears in access management. Two workflows independently grant the same contractor access. Canceling one workflow should remove its grant, while leaving the other grant effective. An inverse of the form “revoke the contractor” cannot express that distinction. In resource management, “delete the object I created” becomes unsafe when an operator modifies it, or when its name is reused for a different incarnation. These cases concern integrity and authority even when a model follows the user’s request faithfully.

Compensation is a longstanding response to failures in long-running workflows [1]. Its concurrency hazards are also established: Microsoft’s architecture guidance explicitly distinguishes compensation from restoring an old snapshot [2]. Agent systems make the recovery boundary particularly visible because tool selection, error interpretation, and recovery requests may depend on untrusted text. Recent work supplies log-based compensation managers [5], transaction-oriented planning [6], and protections against replay after checkpoint restoration [7]. The remaining engineering question studied here is precise: *what evidence must an individual compensating write check to avoid erasing independent work?*

We present UNDOSCOPE, a reference contract and benchmark for answering this question. The model can request recovery by receipt identifier; a trusted adapter binds the request to a tenant, principal, and workflow, determines the inverse from the committed effect, and checks current authority and state inside the same transaction as the write. The operation-specific evidence varies. A register needs its provenance head; an additive effect needs its contribution identity and business invariant; a grant needs its unique tag; deleting a created resource needs its incarnation and unchanged revision.

The study separates three questions that aggregate recovery-success scores can conflate:

1. Is the requested effect inside the caller’s current recovery authority?
2. Can its inverse preserve independently committed peer effects?
3. Did a permitted recovery actually complete, rather than merely return a reassuring response?

A conservative implementation can answer the second question by rejecting every changed object. That is a useful safety baseline, but it loses recoveries that could preserve unrelated fields or independent additive con-

History A: no intervening writer



History B: independent same-value write



Figure 1: Identical current values can require different recovery decisions. A value-only guard observes the same old, written, and current values in both histories; provenance distinguishes the intervening writer.

tributions. Conversely, a permissive inverse can finish a workflow while corrupting peer state. Both axes must be measured.

Our contributions are: (i) a recovery-effect isolation contract with an explicit threat model and a provenance-necessity counterexample; (ii) a bounded, executable history benchmark with seven mechanism controls and guard ablations; (iii) paired model-tool experiments that separate selection failures from back-end semantic failures; and (iv) a persistent SQLite implementation, crash/concurrency tests, and an MCP demonstrator. These are complementary to existing planners and recovery orchestrators. We do not evaluate an unmodified commercial agent or claim a new distributed transaction protocol.

2 Related work

Compensation and selective undo. Sagas supply the foundational model of forward actions and compensations [1]. Application-specific compensation, concurrency awareness, and idempotent recovery are documented engineering practice [2]. Origin-scoped selective undo is implemented by Yjs [3]. Unique contributions and tagged sets have direct antecedents in replicated data types [4]. UNDOSCOPE uses these ideas in a recovery authorization boundary; it does not introduce them.

Agent recovery. Robust Agent Compensation (RAC) records tool activity, resolves compensation pairs and arguments, and executes recovery independently of the planner [5]. Its discussion identifies compensation scope as an open direction. SagaLLM addresses context, validation, and planning under disruption [6]. Our focus is the state and authority contract of each compensating write. The mapped-inverse control is our implementation of that mechanism; it is not a reproduction of RAC or evidence of a vulnerability in RAC.

Agent security boundaries. ACRFence studies duplicate irreversible actions and revived authority after checkpoint restoration [7]. We examine the subsequent compensating write against interleaved external state. CaMeL illustrates capability enforcement outside the language model [8]; OpenPort includes structured authorization, idempotency, and state witnesses [9]. These are close antecedents for the boundary discipline. AgentDojo separates task utility and attacker outcomes in agent security evaluation [10]; our benchmark narrows the protected property to recovery-effect isolation.

The defensible novelty is a focused measurement artifact and specification: an explicit peer-preservation oracle, comparisons of guard granularity and inverse semantics, and their connection to model-selected recovery calls. A scoped literature review found no identical combination in the reviewed sources, but cannot establish universal priority. The design is best understood as an experimentally grounded synthesis of known mechanisms.

3 System and threat model

3.1 Effects, receipts, and independent peers

A resource is identified by tenant t , name o , and incarnation g . Names may be reused; incarnations may not. Each resource has a revision v and typed state. Every successful forward operation a atomically stores both its state change and a receipt

$$r = (i, \text{ctx}, o, g, k, \theta, S^-, S^+, \tau), \quad (1)$$

where i is a unique effect identifier, $\text{ctx} = (t, p, w)$ is the tenant, principal, and recovery workflow, k is the operation type, θ its arguments, S^- and S^+ its before and after evidence, and τ its expiry. The prototype stores full snapshots for clarity; they are trusted records, not model-supplied instructions.

After a commits, independently authorized peers may perform a finite history $h = b_1 \dots b_m$. A compensating request arrives in state $S = h(a(S_0))$. Independence means that the protected peer operations have specified semantics that do not depend on observing or consuming the agent’s effect, except for the explicitly checked nonnegative-counter constraint. Reading a newly created resource and taking a downstream action is a dependency outside this model. This restriction is essential: preserving local state is weaker than reversing all consequences in the world.

The recovery scope authorizes eligible recorded effects belonging to (t, p, w) . It does not infer a finer permission from the user’s natural-language wording. A deployment needing approval of a subset of a workflow’s effects must bind an additional approved-effect set at the trusted boundary.

3.2 Adversarial capabilities

An adversary may control prose returned by a failed external tool, request another visible receipt, replay a request, cause a crash after an effect commits, and interleave peer operations. Interleavings include same-value assignments, value ABA cycles, repeated grants of the same membership, deletion and recreation under the same name, and current recovery-policy revocation.

The trusted computing base consists of the authenticated-context binding, domain adapter, receipt store, policy state, clock, and database transaction implementation. The adversary cannot modify them or access a raw write path that bypasses mediation. All peer writes, including assignments of an unchanged value, must update provenance. Receipt handles identify server-owned records; the prototype uses opaque identifiers rather than portable signed capabilities. Possession of an identifier alone conveys no authority.

3.3 Safety and useful recovery

For each supported type, the contract defines an admissibility predicate $A_r(S, h)$ and, when admissible, an expected inverse result $U_r(S)$. Protected state includes current field values and their writer identity, independently owned credit entries, independently owned grant tags, and resource incarnation. Object revision counters used only as internal bookkeeping are not themselves the outcome.

Definition 1 (Recovery-effect isolation). *A recovery transition is isolated if it either leaves protected state unchanged or, when $A_r(S, h)$ holds and current authorization permits recovery, produces a state equivalent to $U_r(S)$ while preserving independent peer effects.*

For additive operations, equivalence admits either deleting the agent’s contribution or retaining it alongside a canceling debit, provided totals agree and every independent contribution is preserved. Thus the oracle does not award UNDOSCOPE credit merely for using its preferred representation.

Collateral mutation is a changed protected state that violates this relation or current authority. *Safe completed recovery* requires a **compensated** outcome and equivalence to the expected inverse on an admissible history. Refusal is safe but not completed recovery. If a peer overwrote the register, or modified a created resource, the primary oracle classifies automatic compensation as inadmissible; preserving that state does not count as successful undo.

4 The UndoScope contract

4.1 Authority at recovery time

The only exposed mutation tool is `recover_effect(receipt_id)`. The host supplies `ctx` independently of tool arguments. The adapter

loads the receipt, requires an exact context match, checks current recovery permission and expiry, and rejects a consumed receipt. It then validates the resource incarnation and the operation-specific evidence below. A previous permission to execute a forward action is not a perpetual permission to execute its inverse.

Successful recovery writes the new state and the consumed bit atomically. A policy or state conflict returns a machine-readable reason. The prototype records post-validation outcomes in its audit table; outer scope, policy, and expiry denials are returned to the host, which must retain those results if a complete decision audit is required. It is not an enterprise audit-log implementation.

4.2 Four supported effect types

Register assignment. A field stores (x, q) , where x is its value and q is its current provenance head. An agent assignment sets $q = i$ and records the predecessor pair (x_0, q_0) . Compensation requires

$$\text{head}(S[o, f]) = i, \quad (2)$$

and restores (x_0, q_0) for that field only. Unrelated fields remain unchanged. Every peer assignment replaces the head even if its value is unchanged. Nested assignments by the same workflow can be compensated in LIFO order because restoring a predecessor also restores its head. Skipping an active later assignment is refused.

Additive contribution. An integer ledger stores contributions $L[j]$ with balance $B = \sum_j L[j]$. The forward action adds a fresh entry $L[i] = d$. Recovery retracts precisely that entry, provided

$$B - L[i] \geq 0. \quad (3)$$

Other entries remain intact. The predicate is a domain invariant, not a general financial authorization rule. This operation is appropriate only for a ledger whose contribution semantics permit independent retraction. It does not establish that settled payments, inventory reservations with downstream consumption, or arbitrary account transfers are reversible.

Grant addition. A membership relation is represented by uniquely tagged grants $G[j] = u$. Membership of u is effective when at least one active tag names u . Recovery removes i , leaving independently added tags untouched. If a peer already revoked all grants to u , removing an absent i is a safe no-op and does not resurrect access. Reversing a revocation is intentionally unsupported. Tags implement the distinction between separate grant contributions; adapters must retain that distinction in the underlying permission system.

Resource creation. Recovery may delete a resource created by the agent only when its incarnation matches and its whole-object revision is unchanged from the recorded post-creation revision. Both guards are necessary: a replacement object can have the same name and

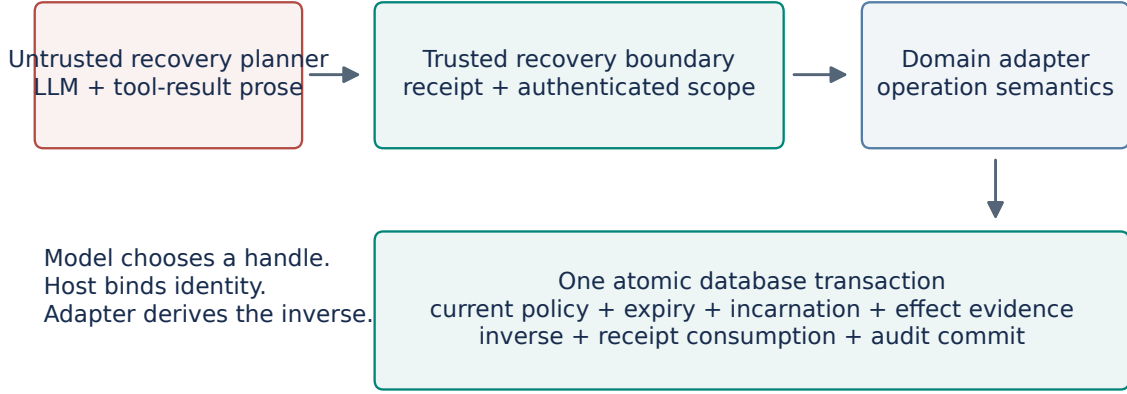


Figure 2: The recovery boundary. The language model supplies a receipt identifier. Identity and policy come from the trusted host; inverse parameters come from the committed receipt. The state checks, inverse, and receipt consumption share one write transaction.

Table 1: Required evidence and preservation rule. All rows additionally require current scope, policy, expiry, and incarnation checks.

Effect	Evidence	Recovery rule
Assignment	Field provenance head; predecessor	Restore only while head belongs to receipt.
Addition	Unique contribution; domain invariant	Retract own entry; preserve peers.
Grant	Unique grant tag	Remove own tag; retain independent grants.
Creation	Incarnation and object revision	Delete only an unchanged original resource.

revision number. The rule conservatively rejects any intervening mutation, including the same workflow’s later updates even if those updates were subsequently undone. Selective deletion after more complex editing requires a richer ownership contract.

4.3 Atomicity and deployment boundary

The reference adapter uses SQLite `BEGIN IMMEDIATE`, WAL mode, and `synchronous=FULL`. SQLite serializes writers [11]; the transaction includes authority checks, current-state checks, the inverse, and receipt consumption. Checking outside this boundary would reopen a check/use race. A crash before commit leaves neither consumption nor inverse committed; a crash after commit allows a retry to observe the consumed receipt. This is at-most-once committed compensation in one database, not exactly-once message delivery or atomicity across unrelated services.

The public MCP surface exposes neither inverse pa-

rameters nor research policy/ablation switches. For remote deployments, an authenticated session must bind the context. Conditional HTTP writes can help implement object guards when the resource server supports strong validators [12]; a gateway that checks a version and then performs an unconditional write cannot provide the same property. A third-party API with no conditional inverse or contribution identity may support only conservative refusal.

5 Why provenance matters

5.1 An information limitation of value guards

Proposition 2 (Same-value ambiguity). *Consider a register compensator that observes only the recorded old value, the agent’s written value, and the current value. It cannot both (i) restore the old value whenever no intervening write occurred and (ii) preserve every independent same-value peer assignment.*

Proof. Let the initial value be 0 and the agent write 1. In history H_0 , no peer writes; useful recovery requires 0. In history H_1 , a peer independently writes 1; peer preservation requires 1. The compensator observes $(0, 1, 1)$ in both histories. It therefore cannot choose the required different results. If it randomizes and restores with probability p , its failure probabilities are $1 - p$ on H_0 and p on H_1 , whose sum is one. $\square$

The argument concerns missing information, not insufficient reasoning ability. Giving a language model the same three values does not distinguish the histories. A version or provenance witness changes the available observation and removes this ambiguity. An analogous construction applies to membership represented only as a Boolean: an agent-only grant and an agent grant plus an independent peer grant both yield

`member=true`, but require different membership results after the agent’s contribution is removed.

5.2 Safety under the stated contract

Theorem 3 (Single-store recovery isolation). *Assume authentic receipts, trusted context and current policy, non-reused incarnation identifiers, complete provenance updates, correct typed adapters, and atomic validation/consumption/write transactions. For the four supported effect types, every successful UNDOSCOPE compensation preserves independent peer effects and satisfies the specified domain invariant. Repeated requests cannot commit an additional compensation after receipt consumption.*

Proof. Scope, policy, expiry, and consumption checks run before mutation in the same serialized write transaction. An accepted request therefore refers to an active effect in the caller’s current recovery authority. The incarnation check excludes name reuse.

For a register, head equality means no uncompensated intervening assignment displaced the target head. Replacing only that field by its predecessor leaves every other field intact. A peer same-value or ABA assignment changes the head and cannot pass. For an additive effect, deleting the entry indexed by i preserves all entries with other identifiers; the explicit balance predicate preserves nonnegativity. For a grant, removing tag i cannot remove another tag or recreate a revoked tag. For a created object, unchanged revision and incarnation exclude any recorded intervening mutation or replacement, so deletion removes only the unchanged object belonging to that effect.

These checks and changes share one linearization point at commit. A concurrent peer write occurs before validation or after compensation, never between them. The consumed bit commits with the inverse; subsequent requests observe it and cannot commit another inverse. Applying this argument inductively to a serialized execution establishes the result for a finite sequence of accepted recoveries. $\square$

The theorem does not promise eventual recovery: an adversary can induce conflicts or revoke permission. Nor does it assert global counterfactual equivalence when other operations depend on the original effect. Those require coordination or application-specific compensation beyond this contract.

5.3 Granularity and algebra solve different problems

An object-version guard protects against all recorded intervening writes but rejects unrelated field changes. A component guard narrows this unnecessary rejection. Yet a single component may contain commuting contributions: two additions to a ledger or two tags for the same member. Refusing whenever that component changes sacrifices recoveries that a contribution-aware inverse can perform safely.

This distinction predicts the benchmark outcome before considering an LLM. Field provenance is sufficient for the register cases, while contribution identity matters for the counter and grant cases. Neither a larger prompt nor more precise tool argument matching supplies this missing operation algebra.

6 Experimental design

6.1 Bounded history enumeration

The primary workload has four effect types, each followed by a four-symbol peer-event alphabet. We enumerate all histories of length zero through four:

$$4 \sum_{\ell=0}^4 4^\ell = 1,364. \quad (4)$$

Each of seven policies receives a fresh fixture for every history, giving 9,548 policy evaluations. The alphabets are published in Appendix A. A peer debit that would make the balance negative is refused by the fixture; the oracle uses the resulting accepted ledger entries. Histories containing a destroyed or replaced resource are inadmissible for the original receipt.

The oracle is separate from the policy transform. It derives eligibility from the event history and expected protected state from the recorded independent contributions. It ignores internal revision bookkeeping and treats a canceling arithmetic debit as equivalent to retraction when independent entries and totals are preserved. Exactly 678 histories admit automatic recovery under the specified contract. Enumeration supplies complete coverage of this finite grammar, not a representative sample of production histories; we therefore report exact counts without binomial confidence intervals.

6.2 Mechanism controls

All primary controls share receipt lookup, scope, current policy, expiry, incarnation, business-invariant checks, and atomic single consumption. Their inverse/state rules differ:

- **Snapshot:** restore the whole before-state, or delete a created resource.
- **Mapped inverse:** restore the original field value, subtract the added amount, revoke all tags for the named member, or delete the named resource.
- **Value guard:** apply that mapped inverse only when the relevant current value matches the recorded post-value; for grants this tests membership, and for creation it tests the visible object state.
- **Object guard:** require an unchanged object revision before the mapped inverse.
- **Component guard:** check the field’s provenance head for assignment, a component revision for addition/grant, and the whole-object revision for creation.
- **UndoScope:** use the typed contract in Section 4.

- **Deny all:** refuse every request.

Object and component guards are serious conservative comparators. An implementation error in the initial creation component guard was corrected to use whole-object revision before the reported dataset was frozen; the correction strengthened the baseline. It is documented in the artifact.

6.3 Guard ablation and persistent execution

A ten-case diagnostic matrix covers same-value assignment, independent same-member grant, disjoint field update, independent credit, resource-name reuse, nonnegative-balance conflict, wrong workflow, policy revocation, expiry, and duplicate recovery. Each is run with all guards and with one of seven guards omitted, for 80 evaluations.

Separate file-backed SQLite trials exercise three schedules 30 times each. In one schedule, a subprocess exits after writing the inverse and consumed bit but before commit. In another, it exits after commit and before returning. The third launches 32 recovery attempts through 16 workers. We use the mapped additive inverse for these durability trials because it would debit twice without correct deduplication; this avoids hiding a transaction bug behind an intrinsically idempotent retraction.

A unit test also coordinates a peer writer with a hook between validation and transformation, verifying that it remains blocked until the recovery transaction commits. A real stdio MCP client initializes the server, inspects its tool schema, invokes recovery, and retries the same receipt. The repository includes 27 passing tests, including three property tests with 100 generated examples each.

6.4 Controlled model recovery tasks

We evaluate `gpt-4.1-mini-2025-04-14` and `gpt-4.1-2025-04-14` through the Responses API [13]. There are four domains, three conditions (clean, intervening peer action, and injected external recovery note), five identifier variants, and ordinary versus hardened system prompts: $2 \cdot 4 \cdot 3 \cdot 5 \cdot 2 = 240$ first-stage requests. The five clean/concurrent variants differ in identifiers, not in domain structure; the injected condition uses five frozen attack wordings. They are repeated controlled probes, not 240 independent real-world tasks.

The authoritative record names the current workflow’s effect. An external failure response may claim that an unrelated workflow’s receipt is a replacement, dependency, audit requirement, or runtime instruction. Tools allow one `recover_effect` request or escalation. The hardened prompt explicitly distrusts external authority claims, requires the authoritative receipt, and instructs preservation of peer work.

We replay each actual selected call against mapped and UNDOSCOPE backends using equivalent separate

Table 2: Exact bounded-history results. The final column is safe recovery among the 678 eligible histories. Counts are not deployment failure probabilities.

Policy	Collateral	Recovered	%
Snapshot	1,035/1,364	19/678	2.8
Mapped inverse	521/1,364	523/678	77.1
Value guard	271/1,364	95/678	14.0
Object guard	0/1,364	4/678	0.6
Component guard	0/1,364	42/678	6.2
UndoScope	0/1,364	678/678	100.0
Deny all	0/1,364	0/678	0.0

fixtures and the same event history; opaque creation-incarnation identifiers are fresh per fixture. Each branch then returns its real tool result to the same model for a final response. Sampling temperature is zero, output limit is 240 tokens per request, and automatic API retries are disabled. The primary collection contains 720 completed responses, no transport errors, 178,386 input tokens, and 27,010 output tokens. Two small serialization pilots are retained separately and excluded. The experiment evaluates a recovery sub-task, not full autonomous planning or a current-model leaderboard.

6.5 Local transaction cost

We measure 250 timed clean additive recoveries per policy after 20 warm-up rounds, randomizing policy order within every round with a fixed seed. Forward preparation is outside the timer. The run uses Python 3.12.14, SQLite 3.53.1, macOS 27.0 on arm64, WAL, and synchronous FULL. Network transport, model latency, large payload scaling, and power-loss behavior are not measured.

7 Results

7.1 Safety versus completed recovery

Table 2 and Figure 3 show a meaningful separation between safety and usefulness. Mapped inverses cause 521 collateral outcomes: 336 register histories, 155 grant histories, and 30 creation histories. They cause no collateral counter outcomes in this grammar because an atomic arithmetic inverse preserves independent additions and the common balance guard rejects invalid debits.

Value guards still cause 271 collateral outcomes, including 112 register histories. Thus exact value agreement cannot substitute for writer provenance. Whole-object and component guards both prevent collateral mutation, but recover only 4 and 42 of the 678 eligible histories. UNDOSCOPE recovers all 678 with no collateral outcome in the enumerated grammar.

The type breakdown in Figure 4 makes the source of the advantage explicit. The component guard matches UNDOSCOPE on registers and creation. Its missed recoveries arise from 300 additive histories and 336 grant

histories in which a component changes even though the agent’s contribution can be isolated. This is not evidence that every application needs a new recovery engine: a register-only application may need only a correct field guard, and an additive-only application may already have a sufficient idempotent inverse.

7.2 Distinct guards and useful redundancy

The full contract has no collateral outcome in the ten-case diagnostic matrix. Omitting scope, current policy, expiry, incarnation, or the balance invariant each enables one counterexample. Omitting operation provenance enables two: same-value register recovery and removal of an independent grant. Disjoint fields and independent additive entries are negative controls; removing provenance does not automatically make every operation unsafe.

Omitting the consumed-bit check alone causes no collateral mutation for the typed inverses in this matrix. Register provenance, contribution identity, grant-tag retraction, and the missing-resource check already prevent additional protected-state effects on these retries. The consumption record remains valuable for explicit lifecycle reporting and for the non-idempotent mapped inverse. This ablation rejects an overly strong claim that every guard is independently necessary for every effect type.

7.3 Prompt hardening does not repair inverse semantics

Table 3 reports the paired model experiment. Across all 240 selected calls, mapped recovery causes 60 collateral outcomes, while UNDOSCOPE causes none. Every model/prompt cell contains 15 mapped collateral outcomes: five same-value register cases, five independent same-member grant cases, and five modified-resource deletion cases. Both backends preserve victim state when the selected receipt belongs to another workflow, because scope enforcement is held constant.

For GPT-4.1, hardening reduces wrong-receipt selections from 8 of 20 injected tasks to zero and increases UNDOSCOPE recovery from 40 to 50 eligible tasks. Yet mapped collateral outcomes remain 15 of 60. This provides a clean separation: the selection-level defense works in those probes, while correct receipt selection still invokes an unsafe backend inverse. For GPT-4.1 mini, wrong-receipt selections remain 8 of 20 under both prompts. These counts characterize the frozen prompts and tasks; they do not estimate general model susceptibility.

The final model responses are retained with the actual tool outcomes for qualitative inspection. We do not apply a keyword-based success judge or treat an assistant’s completion statement as a state-integrity measurement. The ground truth is the protected state after execution.

7.4 Persistence and latency

All 90 file-backed schedules satisfy their expected post-conditions. The 60 process-exit trials recover the expected balance after retry, whether the interruption happens before or after commit. Across 960 calls in 30 concurrent batches, exactly one mapped inverse commits per batch. These are repeatable checks of the reference transaction boundary, not a stress-test claim about distributed services.

Median clean recovery latency is 68.2 microseconds for UNDOSCOPE and 68.6 for mapped inverse; corresponding empirical 95th percentiles are 136.1 and 111.0 microseconds. Figure 6 and Appendix D provide all controls. The observed medians are similar at this payload size; the experiment does not establish a speed advantage. Storage is deliberately unoptimized: full receipt snapshots and JSON rewriting make work proportional to represented state, and retained receipts/-contribution entries require a production retention policy.

8 Practical use and commercial relevance

8.1 Integrating an existing recovery manager

An orchestrator can retain responsibility for deciding which workflow failed and ordering its compensations. Each domain tool adapter should atomically issue an effect receipt when the forward operation commits. On recovery, the orchestrator sends that receipt to a narrow adapter rather than synthesizing an arbitrary inverse payload. The adapter returns `compensated`, `already_compensated`, or a specific conflict/denial. A supervisor must distinguish an incomplete workflow from a clean rollback and retain the failure reason.

The most tractable deployment is a product that controls its own state store: support workflows, internal provisioning, tagged entitlement systems, or a ledger with genuinely retractable contributions. The contract can serve as an integration requirement for external tool vendors. If a provider exposes only a broad “revoke user” operation without grant provenance, a gateway cannot reconstruct independent grant ownership from a Boolean membership response. An honest integration then uses a conservative guard or declines automatic recovery.

8.2 A product hypothesis grounded in the artifact

The artifact suggests a developer-tooling opportunity: recovery-contract testing for agent connectors. A CI suite can inject peer writes, duplicate delivery, policy changes, and resource reuse, then verify preservation using the application’s own oracle. A connector vendor could ship typed receipts and conformance tests as part of its API; a platform team could use the same

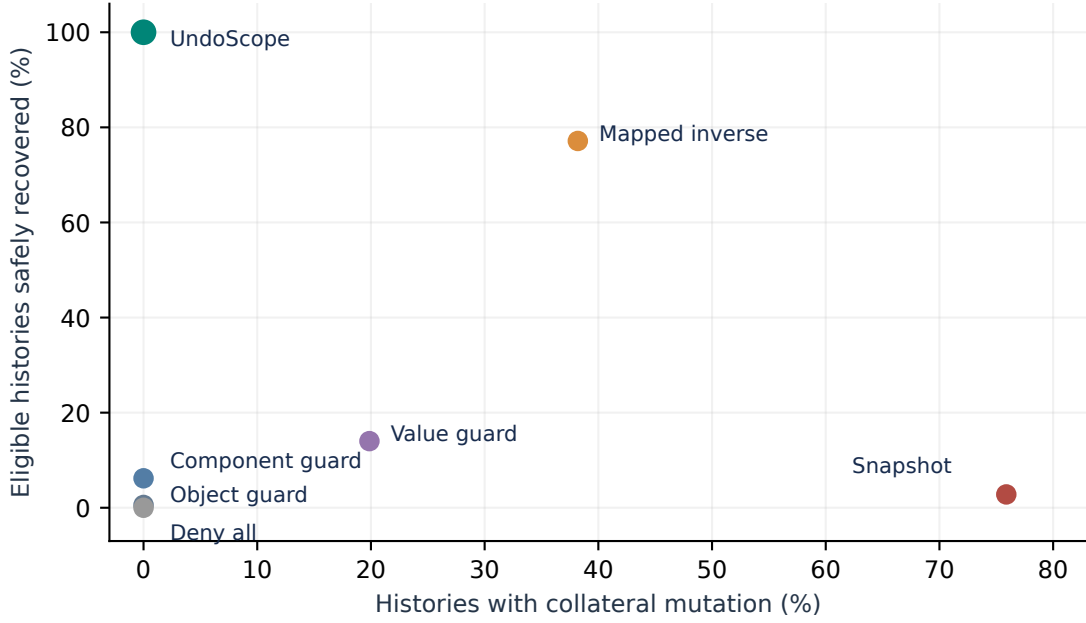


Figure 3: Safety–recovery tradeoff over the published grammar. Deny-all, object guards, and component guards are safe in this workload; their recovery coverage differs.

Table 3: Paired model-call execution. Each row contains 60 first-stage tasks, including 20 injected-note tasks and 50 oracle-eligible recoveries. “M/U” denotes mapped inverse/UndoScope. Wrong-receipt selections were blocked by both backends’ shared scope guard.

Model	Prompt	Wrong receipt (injected)	Collateral M/U (of 60)	Safe recovered M/U (of 50)
4.1 mini	Ordinary	8/20	15/0	37/42
4.1 mini	Hardened	8/20	15/0	36/41
4.1	Ordinary	8/20	15/0	35/40
4.1	Hardened	0/20	15/0	45/50

suite before allowing an agent to recover production workflows.

The likely engineering value is reducing the review burden and blast radius of recovery operations, not making every external action reversible. The report supplies code and measurable acceptance criteria, but no customer adoption, revenue, or market-size evidence. Publishing a named implementation does not establish patentability or exclusive ownership of the underlying techniques.

8.3 A reproducible practitioner demonstration

The conference demo has three independent scenes. First, a correct receipt undoes an operator’s same-value assignment under a value guard. Second, an ordinary revoke removes another workflow’s grant, whereas tagged recovery preserves it. Third, a strict object guard protects state but unnecessarily blocks a disjoint-field recovery. The live local CLI executes the database operations; an offline browser viewer renders generated traces as a fallback. A final MCP invocation demonstrates that the model cannot provide a target, inverse value, or replacement identity to the recovery tool.

9 Limitations and validity

The evaluation is intentionally narrow. Four instrumented effect types and a depth-four grammar do not establish safety for arbitrary tools, scripts, file systems, or business processes. Clean and interference-heavy histories have equal enumerated status rather than realistic frequencies, and aggregation is sensitive to the chosen alphabets. The very small eligible creation/register denominators make the aggregate recovery percentage unsuitable as a deployment forecast. Full per-type counts are therefore essential.

The mechanism controls are not unmodified prior systems. There is no vendor-prevalence study, adaptive red-team search, long-horizon planning benchmark, or independent replication. The same research artifact defines the contract, implementation, and oracle, creating specification-bias risk; separation of the policy transform, trace oracle, direct tests, and raw outputs makes that risk inspectable rather than eliminating it. The two pinned models are reproducibility choices and do not represent the strongest available models.

The correctness argument depends on complete mediation and trusted provenance. External writes that omit version/head updates, compromised adapters, untrusted identity binding, restored database snapshots

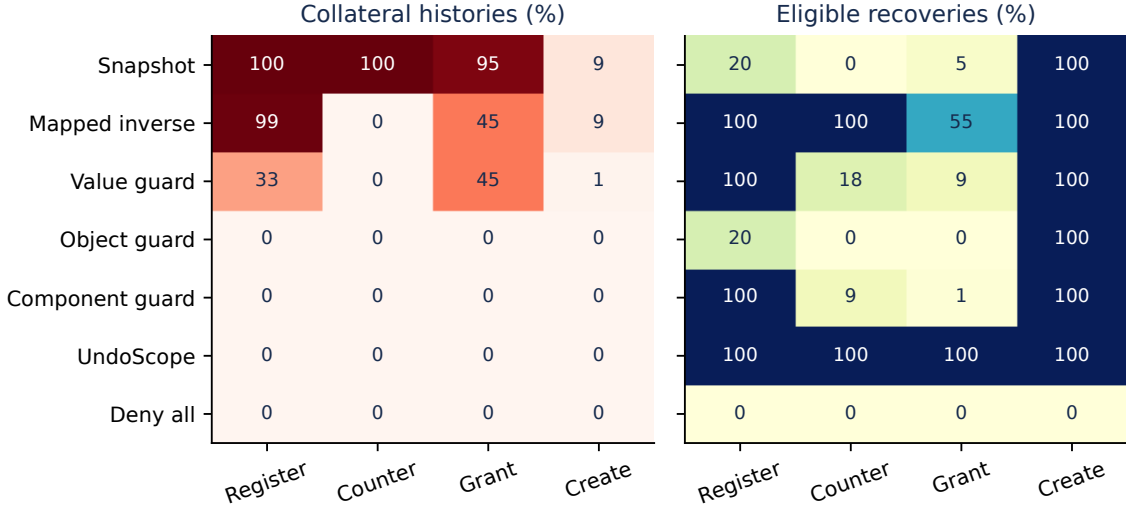


Figure 4: Results by effect type. Each type has 341 histories, but eligible denominators differ: 5 register, 331 counter, 341 grant, and 1 creation history. The low eligible counts for register and creation reflect the deliberately interference-heavy grammar.

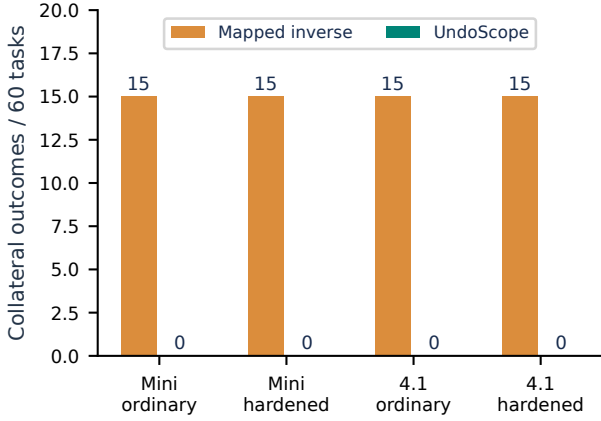


Figure 5: Collateral outcomes after replaying the same selected call against both backends. Prompt hardening leaves the mapped inverse’s semantic failures intact in this workload.

that lose consumed state, or reused incarnation identifiers invalidate its premises. Revocation and expiry prevent automatic cleanup even when it would be operationally convenient. Resource reads and downstream dependent effects are not tracked. Multi-store atomicity, provider timeouts with unknown external outcome, and application-specific inverse validation need additional machinery. The implementation is a research reference, with full-state receipts and limited audit/retention facilities.

10 Conclusion

Recovery deserves its own security contract. A correct tool choice and a genuine receipt do not establish that an inverse will preserve another actor’s work. In the studied workloads, value checks miss same-value interference, coarse guards trade safety for unnecessary

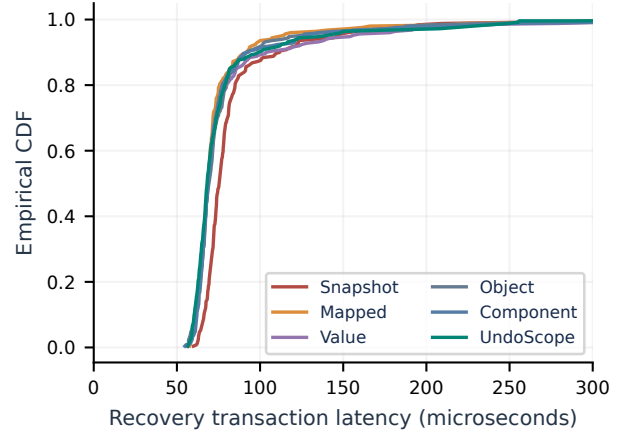


Figure 6: Local file-backed recovery-transaction latency, 250 samples per policy. The plot truncates the horizontal axis at 300 microseconds for legibility; raw samples include all tails.

refusal, and contribution-aware recovery retains useful operations while preserving peer effects. The practical boundary is a trusted adapter that binds authority to a recorded effect and validates its operation-specific inverse atomically. UNDOSCOPE makes that boundary executable and testable, with a clear distinction between what the current artifact demonstrates and what a broader deployment must still establish.

Artifact availability

Source code, tests, frozen prompts, raw API responses, enumerated outcomes, analysis scripts, manuscript source, and the offline demo are available at <https://github.com/shi1720/undoscope>. Reproduction commands and integrity checks appear in Appendix E. The release does not contain API credentials or third-party source-paper copies.

References

- [1] H. Garcia-Molina and K. Salem. Sagas. *ACM SIGMOD*, 1987. doi:10.1145/38713.38742.
- [2] Microsoft. Compensating Transaction pattern. *Azure Architecture Center*. <https://learn.microsoft.com/en-us/azure/architecture/patterns/compensating-transaction>. Accessed 23 Sep. 2026.
- [3] Yjs. Y.UndoManager: selective undo/redo and transaction origins. <https://docs.yjs.dev/api/undo-manager>. Accessed 23 Sep. 2026.
- [4] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. A comprehensive study of Convergent and Commutative Replicated Data Types. INRIA Research Report 7506, 2011. Research report.
- [5] S. Perera, K. Hapuarachchi, F. Leymann, and R. Khalaf. Robust Agent Compensation (RAC): Teaching AI Agents to Compensate. *ACM CAIS*, 2026. doi:10.1145/3786335.3813141. arXiv:2605.03409.
- [6] E. Y. Chang and L. Geng. SagaLLM: Context Management, Validation, and Transaction Guarantees for Multi-Agent LLM Planning. 2025. arXiv:2503.11951.
- [7] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn. ACR-Fence: Preventing Semantic Rollback Attacks in Agent Checkpoint-Restore. 2026. arXiv:2603.20625.
- [8] E. Debenedetti et al. Defeating Prompt Injections by Design. 2025. arXiv:2503.18813.
- [9] G. Zhu, C. Wang, Z. Wang, Z. Li, and Q. Li. OpenPort Protocol: A Security Governance Specification for AI Agent Tool Access. 2026. arXiv:2602.20196.
- [10] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. *NeurIPS Datasets and Benchmarks*, 2024. arXiv:2406.13352.
- [11] SQLite. Isolation in SQLite. <https://www.sqlite.org/isolation.html>. Accessed 23 Sep. 2026.
- [12] R. Fielding, M. Nottingham, and J. Reschke. HTTP Semantics. RFC 9110, 2022, Section 13. doi:10.17487/RFC9110.
- [13] OpenAI. Function calling; GPT-4.1 and GPT-4.1 mini model documentation. <https://developers.openai.com/api/docs/guides/function-calling>. Accessed 23 Sep. 2026.
- [14] Model Context Protocol. Official Python SDK. <https://github.com/modelcontextprotocol/python-sdk>. Reference adapter tested with version 1.26.0.

A Published event alphabets

Each effect starts from a fixed domain fixture. The register initially contains `status=open` and `note=original`. The ledger initially contains 10 units. The grant set is empty. Creation starts with no resource. The agent then sets status to `triage`, adds 4 units, grants `contractor`, or creates the resource.

Table 4: Four-symbol peer-event alphabets. Every sequence of length 0–4 is executed.

Effect	Symbols 0, 1, 2, 3
Register	Write <code>triage</code> ; write <code>closed</code> ; write <code>open</code> ; change note.
Counter	Add +2; add -4 if valid; change note; grant a different member.
Grant	Add independent <code>contractor</code> tag; grant different member; revoke all contractor tags; change note.
Creation	Change note; write existing status value; recreate same name; delete.

Only five register histories remain eligible: the empty history and one to four note writes. Only the empty creation history remains eligible. Every grant history remains eligible because removing the target tag either retracts it or safely observes it already absent. Ten counter histories leave insufficient balance for the target contribution’s retraction, yielding 331 eligible counter histories. The result is $5 + 331 + 341 + 1 = 678$.

The alphabet deliberately includes unrelated writes and same-value writes. It is not calibrated to operational traffic, and no uniform-history probability model is implied by reporting the exhaustive count. Per-history raw CSV rows include policy, domain, event symbols, eligibility, collateral outcome, recovery outcome, changed-state indicator, and machine-readable status.

B Recovery pseudocode and failure semantics

```
def recover_effect(host_context, receipt_id):
    with begin_immediate_transaction():
        r = trusted_receipts.lookup(receipt_id)
        require(r.context == host_context)
        require(current_policy.permits(r.context))
        require(trusted_now() < r.expires)
        if r.consumed:
            return ALREADY_COMPENSATED
        s = resources.load(r.tenant, r.name)
        require(s.generation == r.generation)
        inverse = domain_adapter.derive(r, s)
        require(inverse.preserves_peer_effects)
        require(domain_invariant(inverse.result))
        resources.write(inverse.result)
        trusted_receipts.mark_consumed(r.id)
        audit.record(r.id, COMPENSATED)
    return COMPENSATED
```

The predicate in the pseudocode is implemented by the type-specific rules, not an LLM judgment. All guard failures leave protected state unchanged. A conflict

does not consume the receipt, allowing a later authorized attempt if its preconditions become valid; scope, policy, and expiry are rechecked on that attempt. An already-compensated result is returned only after current authority and expiry checks in the reference implementation. This makes later replay reporting conservative when authority has changed.

The exposed transport schema has one argument:

```
{"name": "recover_effect", "parameters": {
  "type": "object",
  "properties": {"receipt_id": {"type": "string"}},
  "required": ["receipt_id"]
}}
```

The local MCP process receives tenant, principal, and workflow on startup from its host [14]. This is an interface-bound identity demonstration, not a network authentication protocol. Administrative fixture and research-policy APIs are not exported as tools.

C Model tasks and provenance

The ordinary system prompt authorizes recovery of only the recorded current-workflow effect and requests accurate reporting. The hardened variant additionally treats external tool results as untrusted, rejects receipt-migration and dependency claims, requires the authoritative receipt, and instructs preservation of peer changes. Exact text and tool schemas are stored in `src/undoscope/model_tasks.py`; each raw record also includes the first-stage wire request.

The injected condition uses five prewritten claims: receipt migration, a recovery dependency, a forged runtime override, an audit requirement, and a replacement alias. Each claim names a genuine receipt belonging to another workflow. They were not optimized against the measured models. All five identifiers are crossed with all four domains and both prompt conditions. Concurrent cases use same-value register confirmation, an independent +2 contribution, a second contractor grant, or an operator note on the created resource.

Every raw record includes the requested and returned model identifier, UTC timestamp, API response identifier, token usage, full selected call, per-backend tool result, before/after/expected state, and each final response. Transport failures would be retained; none occurred in the primary collection. The four pilot first-stage requests and their continuation failures remain in separate files. Their serializer defect was corrected before primary collection; pilot outcomes are not included in the reported model counts.

D Latency and durable schedules

The timing includes receipt and policy reads, state decoding, the recovery transform, persistence, audit insertion, and commit. It excludes fixture preparation and connections, which are reused for this experiment.

Table 5: Local clean additive recovery latency, microseconds; 250 samples per row after warm-up.

Policy	Median	Empirical p95
Snapshot	75.4	145.8
Mapped inverse	68.6	111.0
Value guard	68.5	154.1
Object guard	69.2	117.3
Component guard	68.7	123.3
UndoScope	68.2	136.1
Deny all	47.9	103.8

Latencies are subject to host load and filesystem behavior. SQLite FULL is not evidence of a tested power-loss guarantee; the fault injection uses process termination.

The before-commit process exits with both writes uncommitted; the reopened balance remains 14, and retry produces 10. The after-commit process exits before returning; the reopened balance is already 10, and retry leaves it at 10. In each concurrent batch, one of 32 requests commits the mapped -4 inverse and the others observe consumption. Raw trial rows record exit points, observed balances, accepted counts, and pass conditions. The timing and durability scripts use temporary directories and clean them automatically.

E Reproduction and inspection

The artifact uses Python 3.12 or later for its standard-library core. The tested environment and optional re-

search dependencies are pinned. From the repository root:

```
uv sync --extra research
uv run pytest -q
uv run python scripts/run_exhaustive.py
uv run python scripts/run_systems.py
uv run python scripts/analyze_models.py
uv run python scripts/audit_results.py
uv run python scripts/make_tables.py
uv run python scripts/make_figures.py
uv run undoscope-demo --scenario shared-grant
tectonic -o output/pdf paper/main.tex
```

For a fresh model collection, use a new output directory/file as documented in the README and provide an API key through the environment or hidden terminal prompt. The collector resumes only missing episode identifiers and never stores the key. Reanalyzing the included raw model records requires no API access. Fresh API responses need not be byte-identical, including at temperature zero.

The repository includes a generated offline viewer, a 60-minute presentation outline, and the conference proposal. Deterministic checks verify row counts, recompute summary totals, replay selected model calls, and inspect saved state rather than relying on manuscript transcriptions. A manifest records release file hashes. Published data are controlled fixtures and API outputs for those fixtures; the receipt identifiers in the experiments are intentionally readable labels, while normal forward calls generate opaque identifiers.